

hmstandardization_challenge_prep

June 1, 2026

1 Image Recon Comparison Dataset

This notebook creates a synthetic dataset for comparing image reconstruction algorithms.

Resting state data recorded with a whole-head ninjaNIRS is augmented with synthetic activations with a known spatial and temporal pattern.

The activations are located below C3 and C4, with a Gaussian profile that decays as a function of geodesic distance. The temporal shape follows a canonical Gamma HRF form.

```
[1]: import random
from pathlib import Path

import cedalion.dataclasses as cdc
import cedalion.dot
import cedalion.models.glm as glm
import cedalion.sigproc.quality as quality
import cedalion.sim.synthetic_hrf as synhrf
import cedalion.vis.anatomy
import cedalion.vis.blocks as vbx
import cedalion.vis.quality
import matplotlib.pyplot as p
import numpy as np
import pyvista as pv
import xarray as xr
from cedalion import units

xr.set_options(display_expand_data=False)
```

```
[1]: <xarray.core.options.set_options at 0x727ee1a5c190>
```

```
[2]: OUTPUT_FOLDER = Path("/home/eike/Projekte/ibslab/40_analysis/
↳img_space_std_comparison")
```

Load the ICBM-152 head model

```
[3]: head_ijk = cedalion.dot.get_standard_headmodel("icbm152")
head_ras = head_ijk.apply_transform(head_ijk.t_ijk2ras)
inflated = cedalion.dot.get_inflated_cortex_surface("icbm152")
```

Load a NinjaNirs-22 resting state dataset

```
[4]: rec = cedalion.data.get_nn22_resting_state()
rec["amp_repaired"] = quality.repair_amp(rec["amp"])
rec["od"] = cedalion.nirs.cw.int2od(rec["amp_repaired"])
```

Data Quality Inspection.

SCI and PSP metrics are calculated for 5s time windows, which are considered clean if both metrics are above given thresholds. A whole channel is considered clean, if 75% of all time windows are clean.

```
[5]: window_length = 5 * units.s
sci_thresh = 0.75
psp_thresh = 0.1
sci_psp_percentage_thresh = 0.75

sci, sci_mask = quality.sci(rec["od"], window_length, sci_thresh)
psp, psp_mask = quality.psp(rec["od"], window_length, psp_thresh)
sci_x_psp_mask = sci_mask & psp_mask

perc_time_clean = sci_x_psp_mask.sum(dim="time") / len(sci.time)
sci_psp_mask = [perc_time_clean >= sci_psp_percentage_thresh]

f, ax = p.subplots(1, 1, figsize=(6, 6))

norm, cmap = cedalion.vis.colors.threshold_cmap(
    "perc_clean_time",
    0,
    1,
    threshold=sci_psp_percentage_thresh,
    higher_is_better=True,
    colors=["#000000", "#DC3220", "#4daf4a", "#22ff00"],
)

cedalion.vis.anatomy.scalp_plot(
    rec["od"],
    rec.geo3d,
    perc_time_clean,
    ax,
    title = "Percentage of clean time",
    cmap=cmap, norm = norm,
)

# Prune channels that do not pass the quality test
rec["od_clean"], drop_list = quality.prune_ch(rec["od"], sci_psp_mask, "all")

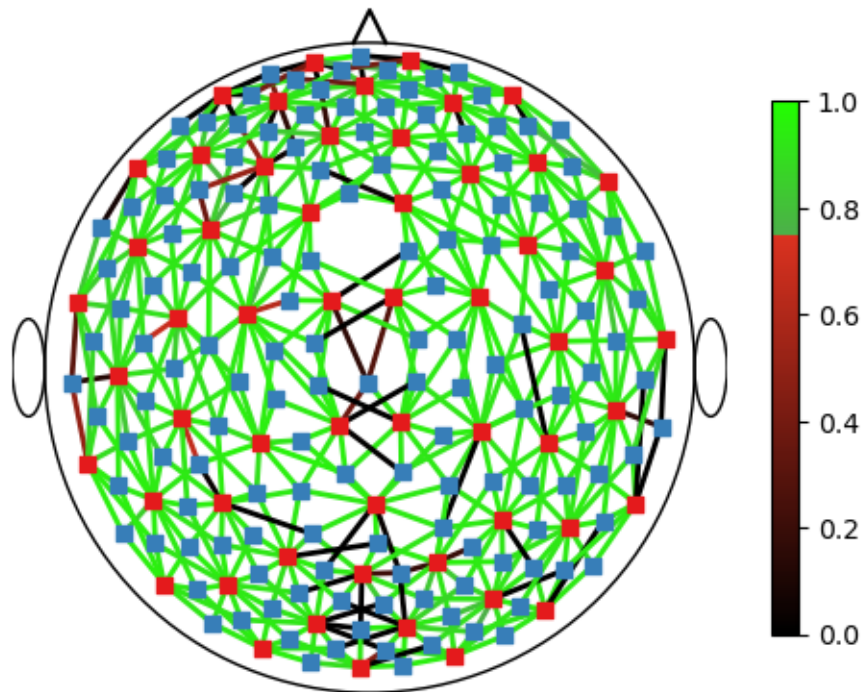
# Display pruned channels
```

```
print(f"List of pruned channels: {drop_list} ({len(drop_list)})")
```

```
OD_NAME = "od_clean"
```

```
List of pruned channels: ['S6D66' 'S6D1' 'S26D3' 'S25D13' 'S5D139' 'S3D68'
'S14D95' 'S2D140'
'S2D36' 'S1D38' 'S1D65' 'S1D67' 'S32D69' 'S32D72' 'S32D67' 'S32D2'
'S31D9' 'S3D66' 'S3D67' 'S4D65' 'S4D36' 'S1D69' 'S2D38' 'S7D39' 'S1D39'
'S6D3' 'S6D70' 'S32D70' 'S40D46' 'S39D46' 'S30D16' 'S35D16' 'S43D56'
'S42D55' 'S45D115' 'S18D115' 'S18D74' 'S16D103' 'S23D103' 'S20D103'
'S43D113' 'S43D55' 'S45D50' 'S45D56' 'S21D78' 'S24D98' 'S19D77' 'S44D115'
'S44D53' 'S44D54' 'S42D116' 'S28D16' 'S23D81' 'S6D17' 'S53D22' 'S49D122'
'S51D122' 'S55D122' 'S49D124' 'S54D130' 'S55D131' 'S51D24' 'S17D112'
'S49D106' 'S48D32' 'S46D127' 'S47D115' 'S47D25' 'S38D126' 'S39D121'] (70)
```

Percentage of clean time



Register montage to scalp

```
[6]: geo3d_relaxed, _ = head_ras.align_and_relax_to_scalp(rec.geo3d, rec["amp"])
```

Visualize registered montage

```
[7]: plt = pv.Plotter()
     vbx.plot_surface(plt, head_ras.scalp, color="w")
     vbx.plot_labeled_points(plt, geo3d_relaxed)
     plt.show()
```

```
Widget(value='<iframe src="http://localhost:41243/index.html?
  ui=P_0x727edf14d650_0&reconnect=auto" class="pyvi...
```

Load precomputed sensitivity matrix. Keep only brain vertices as no activation in scalp tissue is simulated.

```
[8]: Adot = cedalion.data.get_precomputed_sensitivity("nn22_resting", "icbm152")
     Adot_brain = Adot.sel(vertex=Adot.is_brain, channel=rec[OD_NAME].channel)
     Adot_brain
```

```
[8]: <xarray.DataArray (channel: 497, vertex: 25000, wavelength: 2)> Size: 99MB
     2.339e-17 2.339e-17 6.079e-17 6.079e-17 ... 1.661e-24 2.084e-09 2.084e-09
Coordinates:
     parcel      (vertex) object 200kB 'VisCent_Striate_2_LH' ... 'SalVentAttn...'
     is_brain    (vertex) bool 25kB True True True True ... True True True True
     * channel    (channel) object 4kB 'S10D87' 'S10D94' ... 'S47D29' 'S5D137'
     detector    (channel) object 4kB 'D87' 'D94' 'D89' ... 'D129' 'D29' 'D137'
     source      (channel) object 4kB 'S10' 'S10' 'S10' ... 'S56' 'S47' 'S5'
     * wavelength (wavelength) float64 16B 760.0 850.0
Dimensions without coordinates: vertex
Attributes:
     units:      mm
```

1.1 Generate spatial activation patterns

The simulated patterns are Gaussians with their maximum directly below C3 or C4 and decaying as a function of geodesic distance.

```
[9]: # obtain the closest vertices to C3 and C4
     _, c3_seed = head_ras.brain.mesh.kdtree.query(
         head_ras.landmarks.sel(label="C3").pint.magnitude
     )
     _, c4_seed = head_ras.brain.mesh.kdtree.query(
         head_ras.landmarks.sel(label="C4").pint.magnitude
     )

     # create the spatial activations below both landmarks
     spatial_act_c3 = synhrf.build_spatial_activation(
         head_ras.brain,
         c3_seed,
         spatial_scale=2 * cedalion.units.cm,
         intensity_scale=5 * units.micromolar,
```

```

    hbr_scale=-0.4,
)

spatial_act_c4 = synhrf.build_spatial_activation(
    head_ras.brain,
    c4_seed,
    spatial_scale=2 * cedalion.units.cm,
    intensity_scale=5 * units.micromolar,
    hbr_scale=-0.4,
)

# concatenate the two spatial activations along a new dimension
spatial_act = xr.concat(
    [spatial_act_c3, spatial_act_c4], dim="trial_type"
).assign_coords(trial_type=["Stim C3", "Stim C4"])
spatial_act

```

```

[9]: <xarray.DataArray (trial_type: 2, vertex: 25000, chromo: 2)> Size: 800kB
[M] 5.704e-24 -2.282e-24 5.406e-23 ... -1.861e-26 1.158e-12 -4.631e-13
Coordinates:
  * chromo      (chromo) <U3 24B 'HbO' 'HbR'
  * trial_type  (trial_type) <U7 56B 'Stim C3' 'Stim C4'
Dimensions without coordinates: vertex

```

Visualize spatial activation patterns

```

[10]: kwargs = dict(
    cmap="RdBu_r",
    vmin=-5,
    vmax=+5,
    cb_label=r"$\Delta$ HbO /  $\mu$ M",
    title=None,
)

f, ax = p.subplots(2, 4, figsize=(20, 10))

for i, pos in enumerate(["C3", "C4"]):
    for j, chromo in enumerate(["HbO", "HbR"]):
        ax_col = 2 * i + j
        trial_type = f"Stim {pos}"

        ax[0, ax_col].set_title(f"{trial_type} - {chromo}")
        cedalion.vis.anatomy.plot_brain_in_axes(
            rec[OD_NAME],
            head_ras.landmarks,
            spatial_act.sel(trial_type=trial_type, chromo=chromo).pint.to("uM"),
            head_ras.brain,
            ax[0, ax_col],

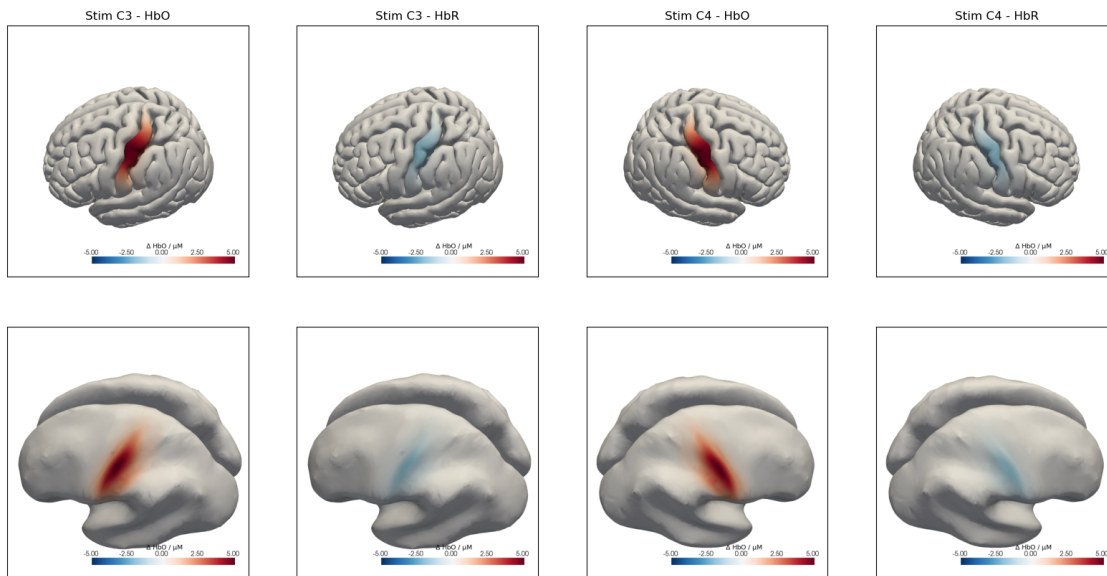
```

```

        camera_pos=pos,
        **kwargs,
    )

    cedalion.vis.anatomy.plot_brain_in_axes(
        rec[OD_NAME],
        head_ras.landmarks,
        spatial_act.sel(trial_type=trial_type, chromo=chromo).pint.to("uM"),
        inflated,
        ax[1, ax_col],
        camera_pos=pos,
        **kwargs,
    )

```



Store the spatial activation patterns in a csv file:

```

[11]: df_spatial_act = spatial_act.pint.to("uM").pint.dequantify().
      ↪to_dataframe("spatial_act")
      display(df_spatial_act)
      df_spatial_act.to_csv(OUTPUT_FOLDER / "spatial_act.csv")

```

	spatial_act		
trial_type	vertex	chromo	
Stim C3	0	HbO	5.704024e-18
		HbR	-2.281610e-18
1	1	HbO	5.405644e-17
		HbR	-2.162258e-17
2	2	HbO	5.416602e-17

...

Stim C4	24997	HbR	-9.352166e-03
	24998	HbO	4.653058e-20
		HbR	-1.861223e-20
	24999	HbO	1.157869e-06
		HbR	-4.631474e-07

[100000 rows x 1 columns]

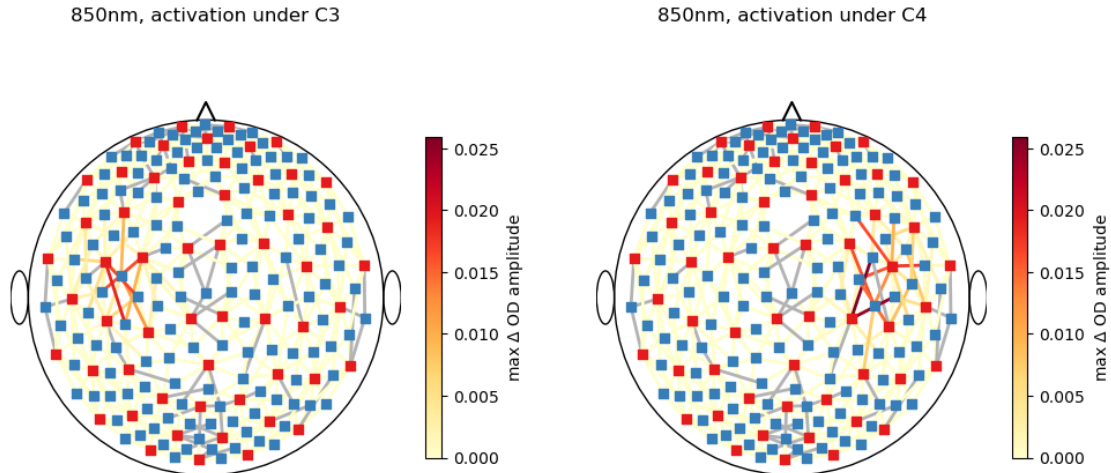
Using the sensitivity matrix, transfer the activation to channel space.

```
[12]: max_od_per_channel = cedalion.dot.forward_model.image_to_channel_space(
        Adot_brain, spatial_act, spectrum="prahl"
    )
max_od_per_channel
```

```
[12]: <xarray.DataArray (trial_type: 2, wavelength: 2, channel: 497)> Size: 16kB
[] -8.025e-12 -2.344e-10 -3.947e-11 -1.243e-12 ... 2.326e-06 1.591e-08 1.811e-08
Coordinates:
* wavelength    (wavelength) float64 16B 760.0 850.0
* channel       (channel) object 4kB 'S10D112' 'S10D133' ... 'S9D94' 'S9D96'
* trial_type    (trial_type) <U7 56B 'Stim C3' 'Stim C4'
    source      (channel) object 4kB 'S10' 'S10' 'S10' 'S10' ... 'S9' 'S9' 'S9'
    detector    (channel) object 4kB 'D112' 'D133' 'D135' ... 'D92' 'D94' 'D96'
```

```
[13]: fig, ax = p.subplots(1, 2, figsize=(12,6))

for i, pos in enumerate(["C3", "C4"]):
    trial_type = f"Stim {pos}"
    wl = 850
    cedalion.vis.anatomy.scalp_plot(
        rec["od"],
        geo3d_relaxed,
        max_od_per_channel.sel(trial_type=trial_type, wavelength=wl),
        ax[i],
        cmap="YlOrRd",
        title=f"850nm, activation under {pos}",
        vmin = 0,
        vmax=max_od_per_channel.max().item(),
        cb_label="max  $\Delta$  OD amplitude",
        optode_labels=False,
    )
```



Find the two channels with the highest activation for activations below C3 and C4

```
[14]: ch_c3 = max_od_per_channel.channel.values[np.argmax(np.abs(max_od_per_channel.
    ↪loc["Stim C3",760,:].values))]
ch_c4 = max_od_per_channel.channel.values[np.argmax(np.abs(max_od_per_channel.
    ↪loc["Stim C4",760,:].values))]
display((ch_c3, ch_c4))
```

```
/opt/miniconda3/envs/cedalion_260507/lib/python3.11/site-
packages/xarray/core/variable.py:315: UnitStrippedWarning: The unit of the
quantity is stripped when downcasting to ndarray.
    data = np.asarray(data)

('S25D28', 'S46D112')
```

1.2 Generate temporal activation patterns

Use a canonical Gamma HRF shape:

```
[15]: basis_fct = glm.Gamma(tau=0 * units.s, sigma=3 * units.s, T=3 * units.s)
```

The dataset contains about 6 min. of resting state data:

```
[16]: rec[OD_NAME].time
```

```
[16]: <xarray.DataArray 'time' (time: 3309)> Size: 26kB
0.0 0.1112 0.2225 0.3337 0.445 0.5562 ... 367.5 367.6 367.7 367.8 367.9 368.0
Coordinates:
    * time      (time) float64 26kB 0.0 0.1112 0.2225 0.3337 ... 367.8 367.9 368.0
    samples     (time) int64 26kB 0 1 2 3 4 5 6 ... 3303 3304 3305 3306 3307 3308
```


Attributes:
units: second

Generate randomly distributed stimulus events covering the whole time range

```
[17]: random.seed(42)

df_stim = synhrf.build_stim_df(
    max_time=rec["od"].time.values[-1] * units.seconds,
    trial_types=["Stim C3", "Stim C4"],
    min_interval=10 * units.seconds,
    max_interval=20 * units.seconds,
    min_stim_dur = 10 * units.seconds,
    max_stim_dur = 10 * units.seconds,
    min_stim_value = 1.0,
    max_stim_value = 1.0,
    order="random",
)

print(f"generated {len(df_stim)} events.")
df_stim
```

generated 14 events.

```
[17]:
```

	onset	duration	value	trial_type
0	16.39	10.0	1.0	Stim C3
1	38.62	10.0	1.0	Stim C4
2	59.49	10.0	1.0	Stim C3
3	84.54	10.0	1.0	Stim C3
4	109.99	10.0	1.0	Stim C4
5	130.05	10.0	1.0	Stim C4
6	151.60	10.0	1.0	Stim C3
7	172.57	10.0	1.0	Stim C4
8	199.87	10.0	1.0	Stim C4
9	225.39	10.0	1.0	Stim C4
10	251.16	10.0	1.0	Stim C3
11	274.05	10.0	1.0	Stim C3
12	296.83	10.0	1.0	Stim C3
13	318.93	10.0	1.0	Stim C4

We can now use our stim dataframe, basis function, and spatial information to create the synthetic HRF timeseries

```
[18]: syn_ts = synhrf.build_synthetic_hrf_timeseries(rec[OD_NAME], df_stim,
    ↪basis_fct, max_od_per_channel)
syn_ts
```

```
[18]: <xarray.DataArray (trial_type: 2, wavelength: 2, channel: 497, time: 3309)>
Size: 53MB
```

```

[] -0.0 -0.0 -0.0 -0.0 -0.0 -0.0 -0.0 -0.0 ... 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
Coordinates:
* wavelength  (wavelength) float64 16B 760.0 850.0
* channel      (channel) object 4kB 'S10D112' 'S10D133' ... 'S9D94' 'S9D96'
* trial_type   (trial_type) <U7 56B 'Stim C3' 'Stim C4'
  source       (channel) object 4kB 'S10' 'S10' 'S10' 'S10' ... 'S9' 'S9' 'S9'
  detector      (channel) object 4kB 'D112' 'D133' 'D135' ... 'D92' 'D94' 'D96'
* time         (time) float64 26kB 0.0 0.1112 0.2225 ... 367.8 367.9 368.0

```

Add resting-state and synthetic time series:

```
[19]: rec["od_augm"] = rec[OD_NAME] + syn_ts
```

```

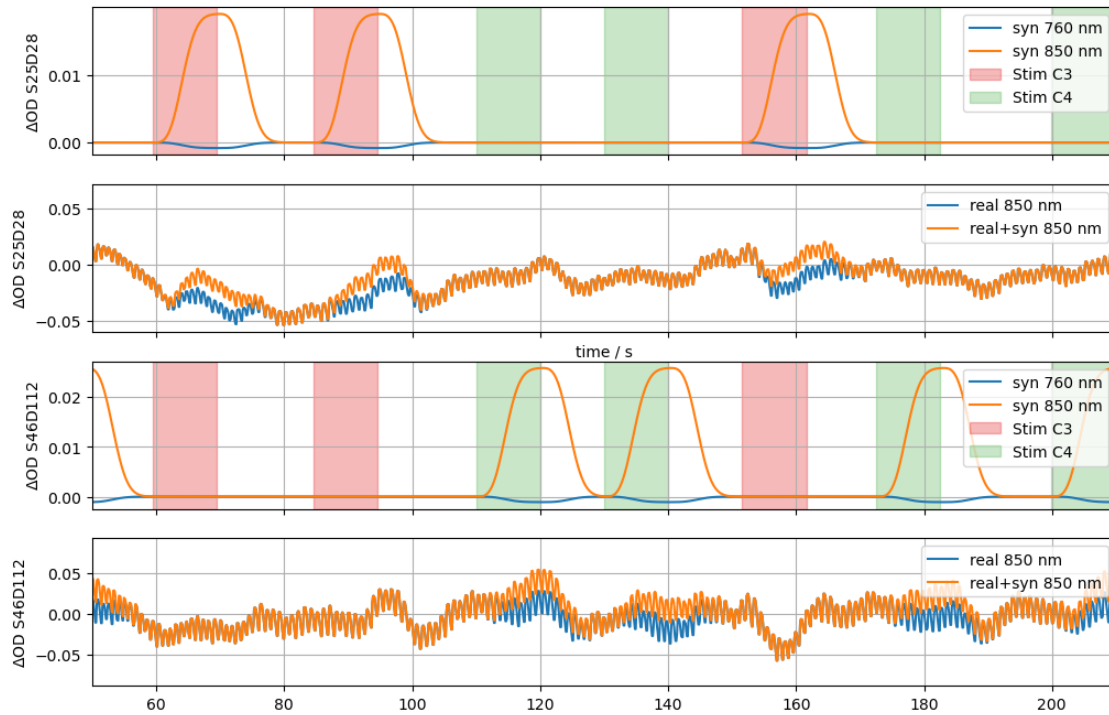
[20]: f, ax = p.subplots(4,1, figsize=(12,8), sharex=True)
for i, (trial_type,ch) in enumerate(["Stim C3", ch_c3), ("Stim C4", ch_c4)):
    for wl in [760., 850.]:
        ax[2*i].plot(syn_ts.time, syn_ts.sel(trial_type=trial_type, channel=ch,
        wavelength=wl), label=f"syn {wl:.0f} nm")

        vbv.plot_stim_markers(ax[2*i], df_stim, y=1.)

        wl = 850.
        ax[2*i+1].plot(rec[OD_NAME].time, rec[OD_NAME].sel(channel=ch,
        wavelength=wl), label=f"real {wl:.0f} nm")
        ax[2*i+1].plot(rec["od_augm"].time, rec["od_augm"].
        sel(trial_type=trial_type, channel=ch, wavelength=wl), label=f"real+syn {wl:.
        0f} nm")

        ax[2*i].legend(loc="upper right")
        ax[2*i+1].legend(loc="upper right")
        ax[2*i].grid()
        ax[2*i+1].grid()
        ax[2*i].set_ylabel(f"$\Delta$OD {ch}")
        ax[2*i+1].set_ylabel(f"$\Delta$OD {ch}")
ax[1].set_xlabel("time / s");
ax[-4].set_xlim(50,210);

```



1.3 Estimating HRF in channel space through block averaging

```
[21]: # apply a frequency filter
rec["od_augm_filtered"] = rec["od_augm"].cd.freq_filter(fmin=0.02, fmax=0.5,
↳ butter_order=4)

# split epochs
epochs = rec["od_augm_filtered"].cd.to_epochs(
    df_stim, # stimulus dataframe
    ["Stim C3", "Stim C4"], # select events
    before=1 * units.seconds, # seconds before stimulus
    after=20 * units.seconds, # seconds after stimulus
)

# calculate baseline
baseline = epochs.sel(reftime=(epochs.reftime < 0)).mean("reftime")

# subtract baseline
epochs_blcorrected = epochs - baseline

# group trials by trial_type. For each group individually average the epoch
↳ dimension
blockaverage = epochs_blcorrected.groupby("trial_type").mean("epoch")
```

```
display(blockaverage)
```

```
<xarray.DataArray (channel: 497, wavelength: 2, reftime: 190, trial_type: 2)>
  Size: 3MB
[] -0.0001992 -0.0001992 -0.0001801 -0.0001801 ... 0.001281 0.001295 0.001295
Coordinates:
  * reftime      (reftime) float64 2kB -0.999 -0.888 -0.777 ... 19.76 19.87 19.98
  * trial_type   (trial_type) <U7 56B 'Stim C3' 'Stim C4'
  * channel      (channel) object 4kB 'S10D87' 'S10D94' ... 'S47D29' 'S5D137'
  * wavelength   (wavelength) float64 16B 760.0 850.0
    source      (channel) object 4kB 'S10' 'S10' 'S10' ... 'S56' 'S47' 'S5'
    detector     (channel) object 4kB 'D87' 'D94' 'D89' ... 'D129' 'D29' 'D137'
```

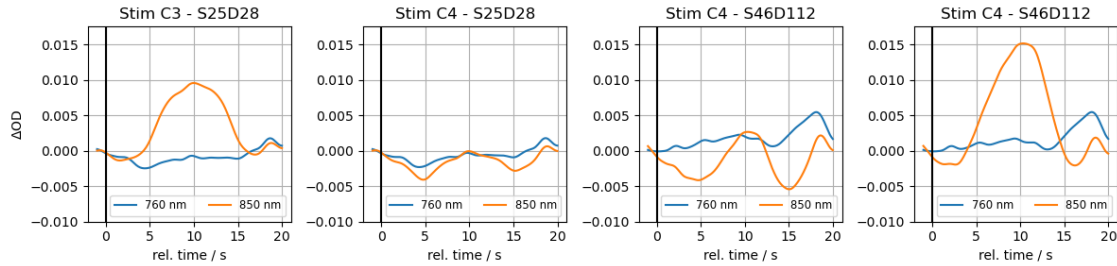
Visualize estimated HRFs

```
[22]: f,ax = p.subplots(1,4, figsize=(12,3))

ax[0].plot(blockaverage.reftime, blockaverage.loc[ch_c3, 760., :, "Stim C3"],
  label="760 nm")
ax[0].plot(blockaverage.reftime, blockaverage.loc[ch_c3, 850., :, "Stim C3"],
  label="850 nm")
ax[1].plot(blockaverage.reftime, blockaverage.loc[ch_c3, 760., :, "Stim C4"],
  label="760 nm")
ax[1].plot(blockaverage.reftime, blockaverage.loc[ch_c3, 850., :, "Stim C4"],
  label="850 nm")
ax[2].plot(blockaverage.reftime, blockaverage.loc[ch_c4, 760., :, "Stim C3"],
  label="760 nm")
ax[2].plot(blockaverage.reftime, blockaverage.loc[ch_c4, 850., :, "Stim C3"],
  label="850 nm")
ax[3].plot(blockaverage.reftime, blockaverage.loc[ch_c4, 760., :, "Stim C4"],
  label="760 nm")
ax[3].plot(blockaverage.reftime, blockaverage.loc[ch_c4, 850., :, "Stim C4"],
  label="850 nm")

ax[0].set_title(f"Stim C3 - {ch_c3}")
ax[1].set_title(f"Stim C4 - {ch_c3}")
ax[2].set_title(f"Stim C4 - {ch_c4}")
ax[3].set_title(f"Stim C4 - {ch_c4}")

for i in range(4):
    ax[i].set_ylim(-0.01, 0.0175)
    ax[i].set_xlabel("rel. time / s")
    ax[i].grid()
    ax[i].legend(loc="lower right", ncol=2, fontsize="small")
    ax[i].axvline(0, c="k")
ax[0].set_ylabel("$\Delta OD")
p.tight_layout()
```



Save these block-average time series into a snirf file:

```
[23]: OUTPUT_FILE = OUTPUT_FOLDER / "synthetic_hrf.snirf"

rec_save = cdc.Recording()
rec_save["hrf_od"] = blockaverage
rec_save.geo3d = geo3d_relaxed
rec_save.stim = df_stim
cedalion.io.write_snirf(OUTPUT_FILE, rec_save)
```

1.4 Image Reconstruction

Reload stored HRFs again:

```
[24]: rec_load = cedalion.io.read_snirf(OUTPUT_FILE)[0]
```

```
[25]: rec_load
```

```
[25]: <Recording | timeseries: ['hrf_od'], masks: [], stim: ['Stim C3', 'Stim C4'],
aux_ts: [], aux_obj: []>
```

```
[26]: rec_load["hrf_od"]
```

```
[26]: <xarray.DataArray (channel: 497, wavelength: 2, trial_type: 2, time: 190)> Size:
3MB
```

```
[] -0.0001992 -0.0001801 -0.0001447 -9.354e-05 ... 0.001278 0.001281 0.001295
```

Coordinates:

```
* time          (time) float64 2kB -0.999 -0.888 -0.777 ... 19.76 19.87 19.98
  samples       (time) int64 2kB 0 1 2 3 4 5 6 7 ... 183 184 185 186 187 188 189
* channel       (channel) object 4kB 'S10D87' 'S10D94' ... 'S47D29' 'S5D137'
  source        (channel) object 4kB 'S10' 'S10' 'S10' ... 'S56' 'S47' 'S5'
  detector      (channel) object 4kB 'D87' 'D94' 'D89' ... 'D129' 'D29' 'D137'
* wavelength    (wavelength) float64 16B 760.0 850.0
* trial_type    (trial_type) <U7 56B 'Stim C3' 'Stim C4'
```

Attributes:

```
data_type_group: processed HRF dOD
```

Estimate channel variances. These will be used to

```
[27]: c_meas = quality.measurement_variance(rec[OD_NAME].pint.dequantify(),
      ↪calc_covariance=False)
      c_meas

[27]: <xarray.DataArray (channel: 497, wavelength: 2)> Size: 8kB
      6.944e-05 0.0002514 0.0001554 0.0004173 ... 0.0006236 0.0001824 0.0005278
Coordinates:
      * channel      (channel) object 4kB 'S10D87' 'S10D94' ... 'S47D29' 'S5D137'
        source      (channel) object 4kB 'S10' 'S10' 'S10' ... 'S56' 'S47' 'S5'
        detector     (channel) object 4kB 'D87' 'D94' 'D89' ... 'D129' 'D29' 'D137'
      * wavelength   (wavelength) float64 16B 760.0 850.0
```

```
[28]: recon = cedalion.dot.ImageRecon(
      Adot.sel(channel=rec_load["hrf_od"].channel),
      recon_mode="mua2conc",
      alpha_meas=0.5,
      brain_only=False,
      spatial_basis_functions=None,
      apply_c_meas=True,
      )

      recon_img = recon.reconstruct(rec_load["hrf_od"], c_meas=c_meas)
```

```
[29]: recon_img.sel(chromo="HbO", trial_type="Stim C3", vertex=Adot.is_brain).max().
      ↪item()
```

[29]:) (micromolar

Visualize reconstruction result on the brain.

```
[30]: for c, channel in [("C3", ch_c3), ("C4", ch_c4)]:
      trial_type = f"Stim {c}"

      f,ax = p.subplots(2,4, figsize=(14,6))
      vmin,vmax = -1,1

      cedalion.vis.anatomy.plot_brain_in_axes(
          rec_load["hrf_od"],
          head_ras.landmarks,
          spatial_act.sel(trial_type=trial_type, chromo="HbO").pint.to("uM"),
          head_ras.brain,
          ax[0,0],
          camera_pos=c,
          cmap="RdBu_r",
          vmin=vmin,
          vmax=vmax,
          cb_label=r"$\Delta$ HbO /  $\mu$ M",
          #title=f"{c} Activation",
```

```

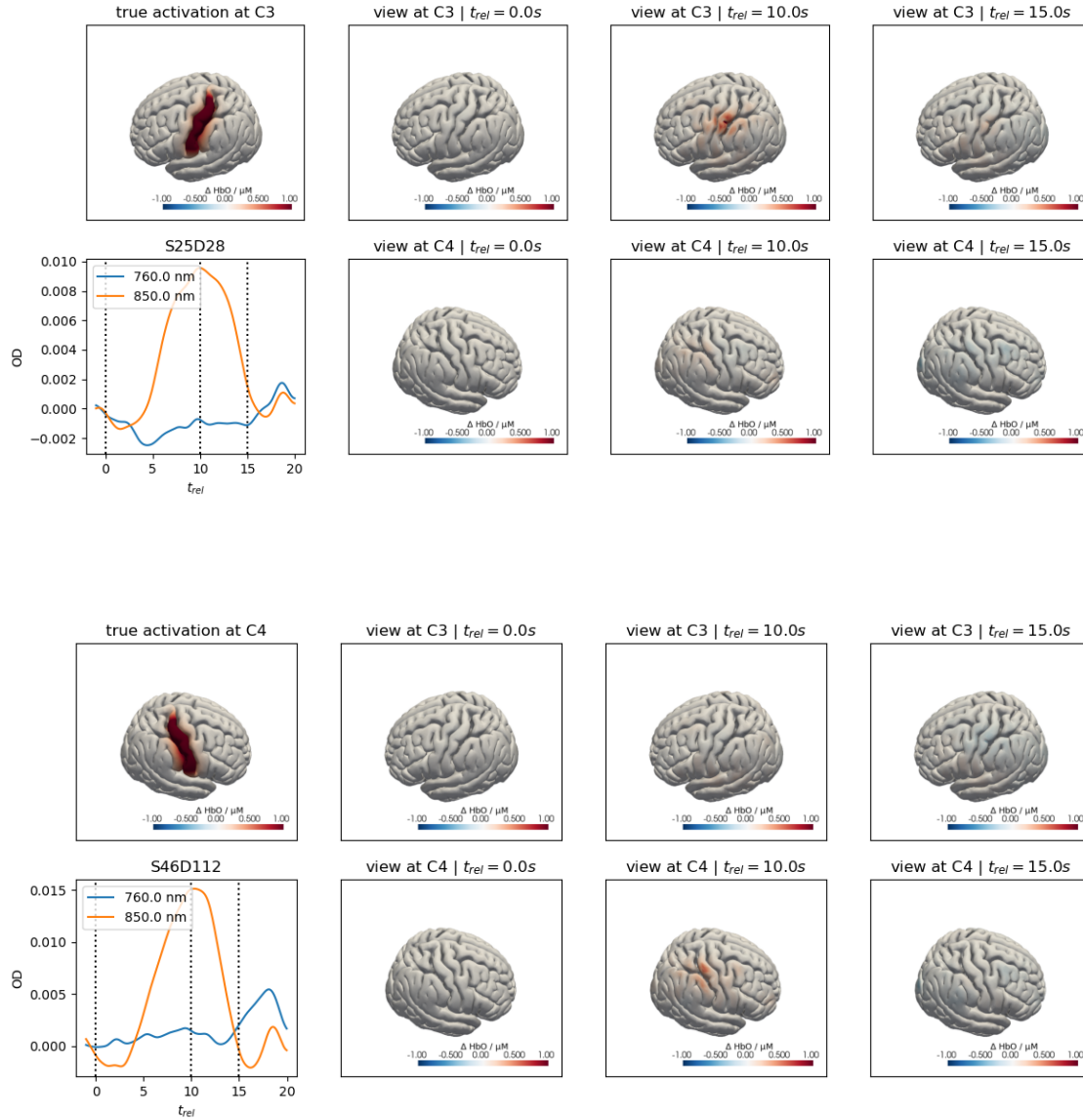
)
ax[0,0].set_title(f"true activation at {c}")

trels = [0., 10., 15.]
for i_wl, wl in enumerate(rec_load["hrf_od"].wavelength.values):
    tmp = rec_load["hrf_od"].sel(
        trial_type=trial_type, channel=channel, wavelength=wl
    )
    ax[1,0].plot(tmp.time, tmp, label=f"{wl} nm")
for trel in trels:
    ax[1,0].axvline(trel, c="k", ls=":")
ax[1,0].legend(loc="upper left")
ax[1,0].set_xlabel("$t_{rel}$")
ax[1,0].set_ylabel("OD")
ax[1,0].set_title(channel)

for i_col, trel in enumerate(trels, start=1):
    for i_row, cam_pos in enumerate(["C3", "C4"]):
        cedalion.vis.anatomy.plot_brain_in_axes(
            rec_load["hrf_od"],
            head_ras.landmarks,
            recon_img.sel(chromo="HbO", trial_type=trial_type, vertex=Adot.
↳ is_brain)

            .sel(time=trel, method="nearest"),
            head_ras.brain,
            ax[i_row, i_col],
            camera_pos=cam_pos,
            cmap="RdBu_r",
            vmin=vmin,
            vmax=vmax,
            cb_label=r"$\Delta$ HbO /  $\mu$ M",
            #title="$t_{rel}$" + f"{trel:.1f} s",
        )
    ax[i_row, i_col].set_title(f"view at {cam_pos} | " + "$t_{rel}$" +
↳ f"{trel:.1f} s")

```



Visualize reconstruction result on the scalp:

```
[31]: for c, channel in [("C3", ch_c3), ("C4", ch_c4)]:
    trial_type = f"Stim {c}"

    f,ax = p.subplots(2,4, figsize=(14,6))
    vmin,vmax = -1,1

    cedalion.vis.anatomy.plot_brain_in_axes(
        rec_load["hrf_od"],
        head_ras.landmarks,
        xr.DataArray(np.zeros(head_ras.scalp.nvertices), dims="vertex"),
```



```

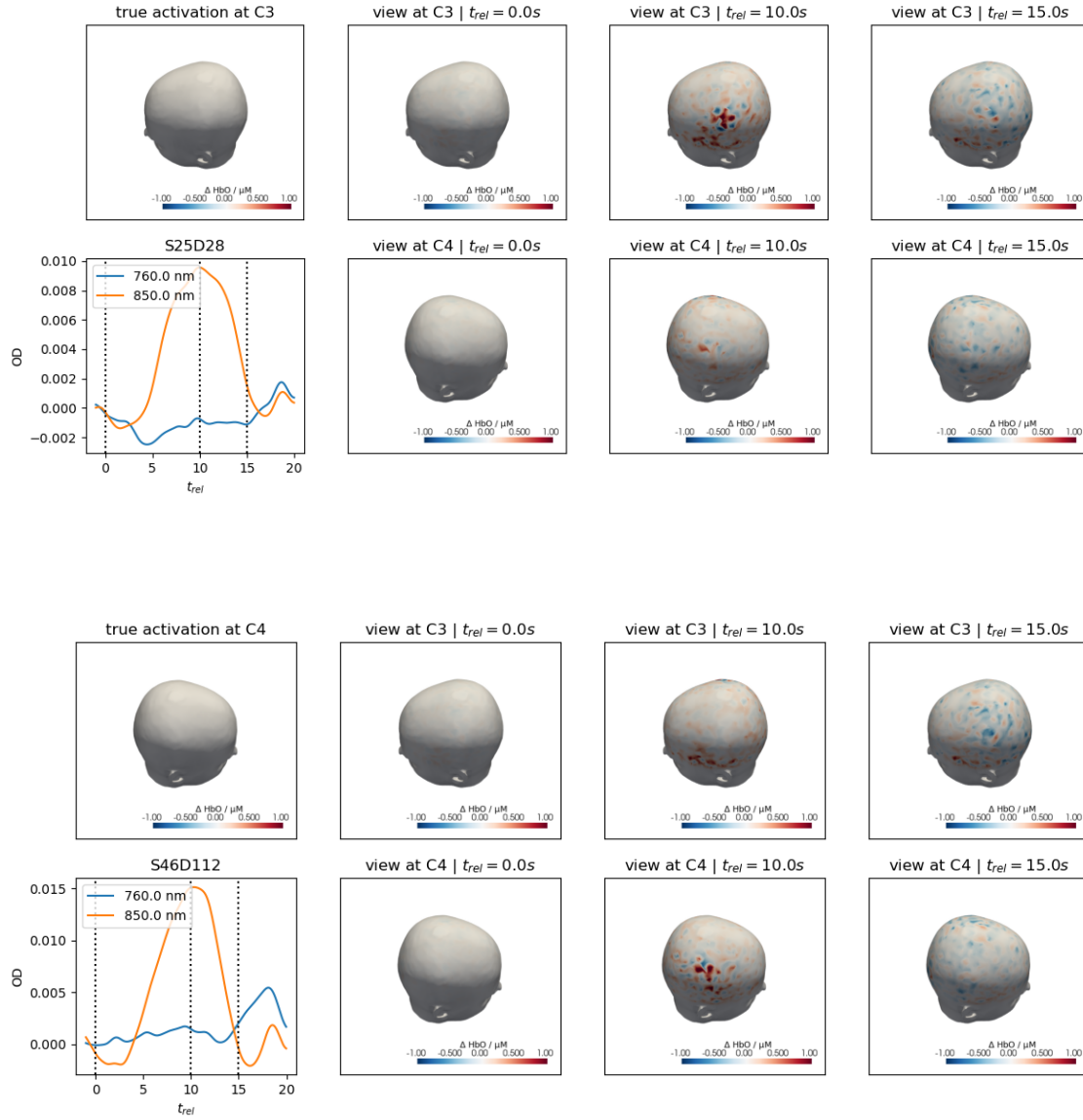
        head_ras.scalp,
        ax[0,0],
        camera_pos=c,
        cmap="RdBu_r",
        vmin=vmin,
        vmax=vmax,
        cb_label=r"$\Delta$ HbO /  $\mu$ M",
        #title=f"{c} Activation",
    )
    ax[0,0].set_title(f"true activation at {c}")

    trels = [0., 10., 15.]
    for i_wl, wl in enumerate(rec_load["hrf_od"].wavelength.values):
        tmp = rec_load["hrf_od"].sel(
            trial_type=trial_type, channel=channel, wavelength=wl
        )
        ax[1,0].plot(tmp.time, tmp, label=f"{wl} nm")
    for trel in trels:
        ax[1,0].axvline(trel, c="k", ls=":")
    ax[1,0].legend(loc="upper left")
    ax[1,0].set_xlabel("$t_{rel}$")
    ax[1,0].set_ylabel("OD")
    ax[1,0].set_title(channel)

    for i_col, trel in enumerate(trels, start=1):
        for i_row, cam_pos in enumerate(["C3", "C4"]):
            cedalion.vis.anatomy.plot_brain_in_axes(
                rec_load["hrf_od"],
                head_ras.landmarks,
                recon_img.sel(chromo="HbO", trial_type=trial_type, vertex=~Adot.
↪is_brain)

                .sel(time=trel, method="nearest"),
                head_ras.scalp,
                ax[i_row, i_col],
                camera_pos=cam_pos,
                cmap="RdBu_r",
                vmin=vmin,
                vmax=vmax,
                cb_label=r"$\Delta$ HbO /  $\mu$ M",
                #title="$t_{rel}$" + f"{trel:.1f} s$",
            )
            ax[i_row, i_col].set_title(f"view at {cam_pos} | " + "$t_{rel}$" +
↪f"{trel:.1f} s$")

```



```
[32]: # store dynamically generated mappings in matrix market format
# import scipy.io
# scipy.io.mmwrite(OUTPUT_FOLDER / "hm_icbm152/voxel_to_vertex_brain.mtx",
#                 ↪ head_ras.voxel_to_vertex_brain)
# scipy.io.mmwrite(OUTPUT_FOLDER / "hm_icbm152/voxel_to_vertex_scalp.mtx",
#                 ↪ head_ras.voxel_to_vertex_scalp)
```